

OBJETIVOS

- Primeiros passos para o desenvolvimento de um SO e testes em um ambiente de máquina virtual.
- Geração de *kernels* executáveis em formatos “.bin” e como ISO.
- Compreensão básica da estrutura de *boot*.
- Operações de I/O em tela em baixo nível.

TAREFA 2: ADICIONANDO FUNCIONALIDADES BÁSICAS AO KERNEL (de 20/01/2025 a 23/01/2025)

Considerando a estrutura da prática de laboratório realizada na TAREFA 1, crie um subdiretório chamado “include”. Dentro desta pasta, serão criados quatro arquivos: `screen.h`, `string.h`, `system.h` e `types.h`. O objetivo destes arquivos é criar e permitir a incorporação de novas funções ao kernel do SO. A partir de agora, as funções serão “habilitadas” (ex. uma função `print`) a partir da inclusão de bibliotecas mediante o uso da diretiva `#include` (como era de se esperar).

Faremos a criação da biblioteca de funções em ordem, começando pelo arquivo `types.h` (Código 7) o qual define os tipos de dados que serão usados nos demais arquivos de biblioteca. As duas primeiras linhas especificam a definição do arquivo de biblioteca. A inserção destas linhas (incluindo o `#endif` ao final) não é uma obrigatoriedade, mas faz parte das boas práticas de codificação de bibliotecas (ex. ao usar a ferramenta CodeBlocks, isso é padronizado ao se criar um arquivo de biblioteca). Na sequência, são definidos alguns tipos inteiros (de 8, 16, 32 e 64 bits) a partir de tipos primitivos de dados. Também foi criado um novo tipo chamado `string`, o qual representa uma cadeia de caracteres sem tamanho definido, a partir de um ponteiro para um tipo de dado primitivo caractere.

```

#ifndef TYPES_H
#define TYPES_H

typedef signed char int8;
typedef unsigned char uint8;

typedef signed short int16;
typedef unsigned short uint16;

typedef signed int int32;
typedef unsigned int uint32;

typedef signed long int64;
typedef unsigned long uint64;

typedef char* string;

#endif

```

Código 7: conteúdo do arquivo *types.h*.

Seguindo a sequência de definição para a biblioteca do SO, faz sentido especificar as funções para manipulação de *strings*. Assim sendo, especificaremos, a seguir o arquivo *string.h*. Por enquanto, este arquivo inclui apenas a função `uint16 strlenh(string)`. A função depende dos tipos `uint16` e `string`, os quais foram definidos em *types.h*. A função descrita no Código 8 retorna um `uint16` (inteiro sem sinal de 16 bits) e recebe como parâmetro uma `string ch`. **Pergunta-se: o que a função `uint16 strlenh(string)` realiza?**

```

#ifndef STRING_H
#define STRING_H

#include "types.h"
uint16 strlenh(string ch)
{
    uint16 i = 1;
    while(ch[i++]);
    return --i;
}

#endif

```

Código 8: conteúdo do arquivo *string.h*.

O próximo arquivo criado é o *system.h* (Código 9). O Código 9 apresenta duas funções uma retorna um valor `char` (`uint8`) e a outra não retorna valor (`void`). Para que possamos usar os tipos de dados que definimos anteriormente em *types.h*, incluímos este arquivo. A primeira função, `uint8 inportb (uint16)` recebe como parâmetro um inteiro de 16 bits e devolve um inteiro de 8 bits. O parâmetro de entrada é um endereço de porta (`_port`) de 16 bits. Internamente a função, a grosso modo, a variável `rv` será conectada a porta `_port` e o valor contido na porta será inserido nesta variável.

Talvez você se pergunte, por que precisamos deste código em Assembly. Por exemplo, se desejarmos controlar onde o cursor aparece em nossa tela, é necessário “falar” diretamente com as

portas. Como existem dispositivos com os quais se deseja conectar (i.e. ler/escrever dados a partir deles), é necessário se usar portas. Na prática, o Código 9 é muito importante porque não é possível se falar com portas diretamente usando linguagem C e é por essa razão que foi criada uma função na linguagem Assembly.

Em alguns dispositivos, como a tela (*screen*), é possível efetuar a comunicação bastando-se escrever dados para uma área específica da memória (RAM). Esta continuará atualizando seus dados dependendo do que está guardado naquela área particular de memória. No caso do cursor, mouse e teclado, é necessário de fato se falar diretamente com as portas. Assim a função `uint8 inportb(uint16)` irá ler a partir da porta e a função `void outportb(uint16, uint8)` irá escrever dados na porta. No caso da função `outportb(uint16, uint8)` o primeiro argumento é a porta desejada e o segundo argumento são os dados que serão escritos para aquela porta particular.

Agora, trataremos dos aspectos mais técnicos do Código 9. Os prefixos e sufixos “__” (dois sublinhados) não chegam a ser necessários, entretanto, podem prevenir conflitos de nomes em programas. A notação apresentada é o que se chama de *extended asm* ou Assembly estendido. Esse é um código Assembly que pode ser usado diretamente dentro de código C para fins de acesso a registradores ou acesso a instruções de baixo nível.

```
#ifndef SYSTEM_H
#define SYSTEM_H
#include "types.h"

uint8 inportb (uint16 _port)
{
    uint8 rv;
    __asm__ __volatile__ ("inb %1, %0"
                          : "=a" (rv)          /* operando de saída */
                          : "dN" (_port)); /* operando de entrada */
    return rv;
}

void outportb (uint16 _port, uint8 _data)
{
    __asm__ __volatile__ ("outb %1, %0"
                          :                     /* operando de saída */
                          : "dN" (_port), "a" (_data)); /* operando de entrada */
}

#endif
```

Código 9: conteúdo do arquivo *system.h*.

O código *assembly* estendido permite a especificação de registradores de entrada, de saída e *clobbered* (podem ser sobrescritos durante a execução). No Código 9, este foi identado de modo a demonstrar cada uma das linhas de entrada/saída/*clobbered* (não usado). A primeira linha que começa com um ‘:’ especifica os operandos de saída, a segunda linha indica os operandos de

entrada e , caso existisse, a terceira linha especificaria os operandos *clobbered*. ‘a’ é um exemplo de constante¹. Constantes de saída devem possuir como prefixo um ‘=’, como é o caso do “=a” (= é um modificador indicando *write only*). Constantes de entrada e saída devem possuir seu argumento em C correspondente entre parêntesis (não vale para a linha de parâmetro *clobbered*). Note o uso de “0” e “1”. Estes são usados para assegurar que quando uma mesma variável é indicada em mais de um lugar no *asm* estendido, esta variável é mapeada apenas para um registrador. Em caso de mais de uma referência a mesma variável, o compilador pode ou não colocá-la no mesmo registrador já atribuído.

As funções `inb` e `outb` são funções `__asm__` para a realização de acesso a entrada e saída (I/O)². No Código 9, na linha de código `__asm__` estendido especificando `inportb`, é possível verificar a leitura do valor partir de `_port` e a atribuição deste a `rv`. A diretiva `__asm__` `__volatile__` foi usada devido aos mecanismos de otimização do compilador. Graças a essas otimizações, o código pode ou não aparecer na localidade especificada pelo programador. Para evitar isso, usa-se essa diretiva, ao invés de simplesmente `__asm__`.

O arquivo `screen.h` introduz as principais funções para manipulação de caracteres na tela do computador. Essas funções são previstas para monitores VGA, que é o padrão usado durante o *boot* de qualquer sistema operacional (Figura 1).

0xb8000				0xb8002												
0		1		2		3		156		157		158		159		
0	‘H’	0x04	‘E’	0x02												
1																
24					Colunas = 80 (cada uma é dividida em 2 bytes, um para caractere e outro para cor). Lihas = 25											

Na Figura 1 é possível ver o endereço da primeira posição na tela (0xb8000). Também é possível verificar que cada posição na tela é definida por dois bytes: o primeiro define o caractere a ser impresso e o segundo define a sua cor. São definidas ao todo 25 linhas por 80 colunas. Os códigos da biblioteca `screen.h` apresentados no Código 10, levam em consideração as definições da Figura 1.

A biblioteca `screen.h` (Código 10) é a consagração de todo o trabalho realizado até o momento. Esta biblioteca utiliza as demais bibliotecas implementadas até então (`types.h`, `system.h` e `string.h`). As principais variáveis são `cursorX` e `cursorY`, as quais representam a posição do cursor na tela (nos eixos x e y). Além disso, as constantes `sw`, `sh` e `sd` representam a largura, altura e profundidade da tela. Percebam, comparando a Figura 1 que a largura é equivalente a quantidade de colunas na tela, a altura é equivalente a quantidade de linhas e a profundidade representa a definição de cada caractere na tela (valor e cor).

Para a biblioteca `screen.h` foram definidas as funções `void clearLine(uint8, uint8)`, `void updateCursor(void)`, `void clearScreen(void)`, `void scrollUp(uint8)`, `void newLineCheck(void)`, `void printch(char)` e `void print(string)`.

```

#ifndef SCREEN_H
#define SCREEN_H
#include "types.h"
#include "system.h"
#include "string.h"

int32 cursorX = 0, cursorY = 0;
const uint8 sw = 80, sh = 25, sd = 2;

void clearLine(uint8 from, uint8 to)
{
    uint16 i = sw * from * sd;
    string vidmem = (string)0xb8000;
    for (i; i < (sw * (to + 1) * sd); i++)
    {
        vidmem[i] = 0x0;
        vidmem[i+1] = 0x0F
    }
}

void updateCursor()
{
    unsigned temp;
    temp = cursorY * sw + cursorX;

    outportb(0x3D4, 14);
    outportb(0x3D5, temp >> 8);
    outportb(0x3D4, 15);
    outportb(0x3D5, temp);
}

void clearScreen()
{
    clearLine(0, sh-1);
    cursorX = 0;
    cursorY = 0;
    updateCursor();
}

void scrollUp(uint8 lineNumber)
{
    string vidmem = (string)0xb8000;
    uint16 i = 0;

    for (i; i < sw * (sh - 1) * sd; i++)
    {
        vidmem[i] = vidmem[i + sw * sd * lineNumber];
    }

    clearLine(sh - 1 - lineNumber, sh - 1);

    if ((cursorY - lineNumber) < 0)
    {
        cursorY = 0;
        cursorX = 0;
    }
    else
    {
        cursorY -= lineNumber;
    }
    updateCursor();
}

```

```

void newLineCheck()
{
    if(cursorY >= sh-1)
    {
        scrollUp(1);
    }
}

void printch(char c)
{
    string vidmem = (string)0xb8000;

    switch(c)
    {
        case (0x08):
            if(cursorX > 0)
            {
                cursorX--;
                vidmem[(cursorY * sw + cursorX)*sd] = 0x00;
            }
            break;
        case (0x09): // Representa a tecla TAB
            cursorX = (cursorX + 8) & ~(8 - 1);
            break;
        case ('\r'):
            cursorX = 0;
            break;
        case ('\n'):
            cursorX = 0;
            cursorY++;
            break;
        default:
            vidmem[((cursorY * sw + cursorX)) * sd] = c;
            vidmem[((cursorY * sw + cursorX)) * sd + 1] = 0x0F;
            cursorX++;
            break;
    }

    if (cursorX >= sw)
    {
        cursorX = 0;
        cursorY++;
    }
    newLineCheck();
    updateCursor();
}

void print(string ch)
{
    uint16 i = 0;

    for(i; i < strlen(ch); i++)
    {
        printch(ch[i]);
    }
}
#endif

```

Código 10: conteúdo do arquivo *screen.h*.

A função `void clearLine(uint8, uint8)`, limpa a tela partir de uma linha específica até outra linha específica. Esta função utiliza uma variável auxiliar `i`, que define o ponto de partida em função de largura (i.e. colunas) e profundidade, para a posição pretendida (`from`). A

variável `vidmem` é uma `string` a qual é inicializada com o valor `0xb8000` (endereço da primeira posição válida na tela). Como se trata de um endereço numérico, faz-se um *casting* por segurança. Na sequência, todas as posições até `to+1` (lembre-se que começamos em 0) são limpas em valor e cor (valor 0 e cor preta, também 0).

A função `void updateCursor(void)`, envia a localização do cursor para uma porta específica (`0x3D4` e `0x3D5`). Estas portas são padronizadas para a arquitetura x86 e controlam a posição do cursor na tela em modo VGA (modo usando 80 colunas x 25 linhas x 16 cores). Assim, para mudar a posição do cursor, basta atualizar os valores de `x` e `y` e então chamar a função `void updateCursor(void)`. Neste momento, pede-se ao aluno que salve seu trabalho e tente utilizar a função `updateCursor()` no arquivo `kernel.c`. Compile e execute o seu trabalho usando o `qEmu` e verifique o resultado. Utilize os valores de `cursorX = 20` e `cursorY=15` e teste.

Explicando esta função um pouco melhor, a variável `temp` recebe a posição em função dos eixos `x` e `y`. A função `outportb()` já foi explicada, e em `outportb(0x3D4, 14)`, seleciona a localização do cursor no registrador de controle (Control Register ou CRT). Em `outportb(0x3D5, temp >> 8)`, ela envia o byte de alta no barramento. Em `outportb(0x3D4, 15)`, seleciona o envio do byte de baixa no CRT e em `outportb(0x3D5, temp)` envia o byte de baixa da localização do cursor no barramento.

A função `void clearScreen(void)` faz a limpeza da tela da primeira a última linha e atualiza a posição do cursor nos eixos `x` e `y` para 0. A função `void scrollUp(uint8)` desloca o texto que aparece na tela `lineNumber` linhas. A função `void printch(char)` imprime um caractere na tela e a função `void print(string)` imprime uma `string` na tela. A função `void newLineCheck(void)` é uma função de verificação para confirmar se alcançamos a última linha na tela, tomando as medidas cabíveis.

Como parte de sua última atividade, insira comentários detalhados para as funções `void clearScreen(void)`, `void scrollUp(uint8)`, `void printch(char)`, `void print(string)` e `void newLineCheck()`. Utilize estas funções no contexto de `kernel.c` demonstrando o funcionamento de cada uma delas.

Como observação final, vale destacar que todas as funções declaradas são consideradas "estáticas". Estática aqui não deve ser interpretado como a maneira como se codificam funções em linguagens de alto nível. "Estática" aqui, significa dizer que, da maneira como as funções foram codificadas, se qualquer uma delas for chamada dentro do *kernel* dez vezes, ela será escrita e reescrita internamente ao *kernel* dez vezes. Isso tem o potencial para gerar uma grande quantidade de código repetido. O que irá refletir também no tamanho do executável final. Para otimizar isso,

recomenda-se declarar todas as funções com "extern" e implementá-las em um código ".c" separado (gerando um código objeto próprio), ao invés de inseri-las dentro do arquivo ".h". A vantagem de se organizar o código desta forma é que, toda vez que a função for chamada, não é necessário reescrevê-la. Basicamente, o ambiente de tempo de execução irá procurá-la no lugar onde ela está definida (i.e. seus binários), e a executa sem repetição de código. Ao adicionar tal função ".c", será necessário introduzir sua compilação no arquivo `build.sh` de maneira análoga aos demais códigos que lá estão.

Entregue no SIGAA todos os códigos e scripts resultantes desta tarefa para análise.

REFERÊNCIAS

DUARTE, G. How Computers Boot Up. Many But Fine – Tech and science for curious people. Available: <https://manybutfinite.com/post/how-computers-boot-up/> (Access: 04/10/2022).

@0xAX. Linux-insides - A book-in-progress about the linux kernel and its insides. Available: <https://0xax.gitbooks.io/linux-insides/content/Booting/linux-bootstrap-1.html> (Access: 04/10/2022).